

iReflect DevOps

Production Environment

OS: CentOS 7

Package Manager: yum

Cron Jobs

<https://steemit.com/centos7/@iphuong/how-to-create-cronjob-in-centos-7>

Manage cron jobs:

```
crontab -e
```

Setting up Development Environment

Environment files:

- Local Backend
 - backend/.env.backend.dev
- Local Backend/DB (Docker-Compose)
 - backend/.env.backend.development.local
 - backend/.env.db.development.local
- Production
 - .env.backend.development.local
 - .env.db.development.local

Local Development (without docker):

- Precise Instructions for starting up can be found in respective frontend/backend READMEs in repo
- Local React:

```
yarn dev
```

- Local Django

```
python manage.py runserver 8080
```

- Local Postgres (Loading dump file)

```
cat prod_dump.sql | psql -U <superuser> -d postgres
```

(sql file is filled with list of SQL commands to reconstruct db. We print to output and run them after logging into postgres (another db) using a <superuser> acc (which is not ireflectadmin))

Note: For mac users, the sql file is generated in Linux environment, so the locale "en-US.utf8" must be changed to "en-US.UTF-8" before being able to run.

```
cat dump_2024-04-07_02_56_59_mac.sql | psql -U szelong -d postgres
```

Local Development (with docker):

- To load postgres db dump data into db container:

- ```
cat <dumpfile>.sql | docker exec -i ireflect-db-1 psql -U ireflectadmin -d postgres
```

- Standard docker-compose commands to start up local docker-compose system

```
docker-compose up --build
```

## Docker Commands



Note: The following commands

With the GitHub Action setup to automatically build and push the containers to the Docker Registry, the following commands to build manually in the local environment and push to the online registry is no longer needed and just for reference.

Build Production containers and Push to Online Registry

```
docker-compose -f docker-compose.production.yml build docker push szelongq/ireflect-reverse-proxy:production docker push szelongq/ireflect-frontend:production docker push szelongq/ireflect-backend:production
```

Making Backup of PostgreSQL DB:

```
// Creating dump file docker exec -t your-db-container pg_dumpall -c -U postgres > dump_`date +%Y-%m-%d"_%H%M%S`.sql // Restoring from dump file cat your_dump.sql | docker exec -i your-db-container psql -U postgres
```

## Connecting to Production

VPN: Follow instructions in <https://dochub.comp.nus.edu.sg/cf/guides/network/vpn>, FortiClient

```
ssh sadm@rubric-i.comp.nus.edu.sg
```

```
su root
```

## Backing up before Deployment

1. SSH into production environment

```
ssh sadm@rubric-i.comp.nus.edu.sg
```

2. Generate backup file

```
// Creating dump file docker exec -t pm-tracker-db-1 pg_dumpall -c -U ireflectadmin > dump_`date +%Y-%m-%d"_%H%M%S`.sql
```

3. Exit back to home environment, then download backup file.

```
scp adm@rubric-i.comp.nus.edu.sg:dump_2024-11-13_03_47_23.sql .
```

4. SSH into production and proceed with deployment

## In case of Rollback

1. If production local data is corrupted, transfer backup file from home environment

```
scp dump_2024-07-16_17_20_38.sql adm@rubric-i.comp.nus.edu.sg:dump_2024-07-16_17_20_38.sql
```

2. Restore data from backup file

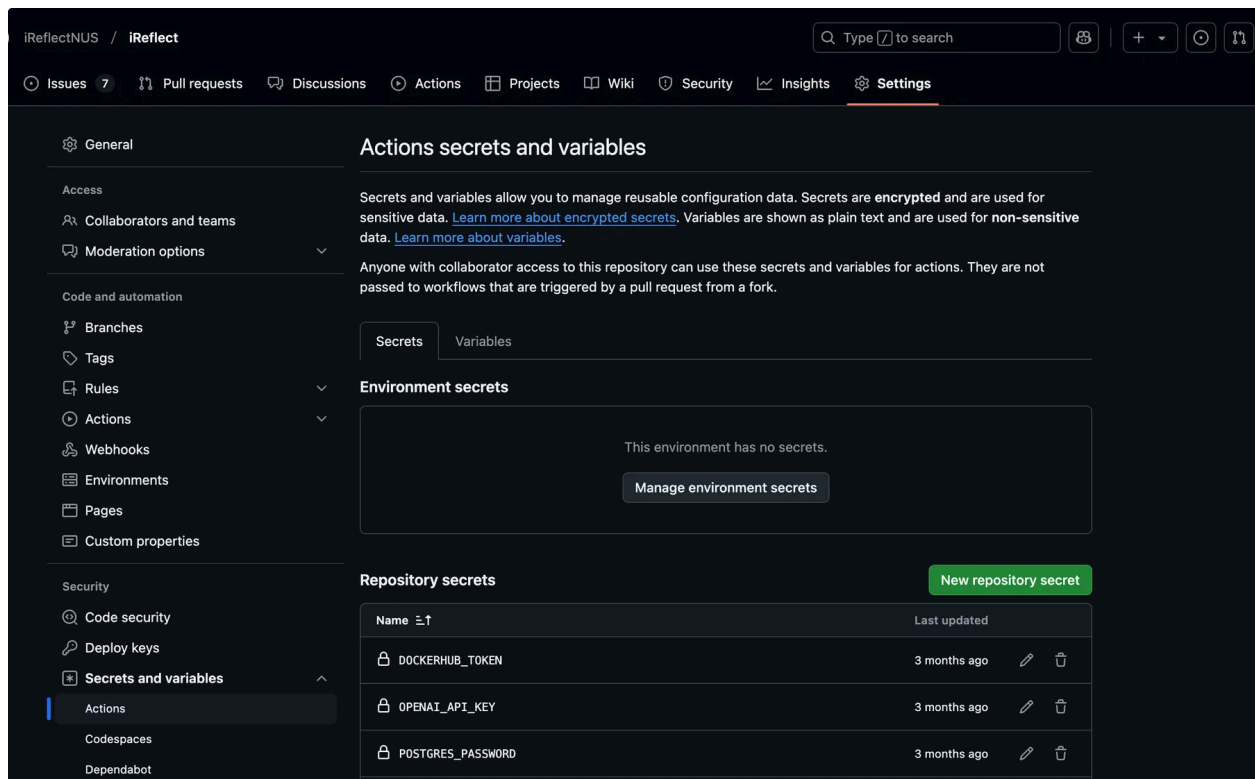
```
// Restoring from dump file cat dump_2024-08-24_00_52_24.sql | docker exec
-i pm-tracker-db-1 psql -U ireflectadmin -d postgres
```

more filename.txt

## Deployment

If there are updates to the environment variables (e.g. API keys), make sure to update the secrets or variables in the iReflect git repo settings before merging PR and building the container.

Secrets for sensitive info like API keys, Variables for non-sensitive info like port numbers and urls



if there are updates to docker-compose environment variables and config files, push the updated env or YAML file to the VM:

```
.env file for docker-compose scp ~/.env.backend.production.local sadm@rubric-i.comp.nus.edu.sg:~/PM-Tracker/.env.backend.production.local # docker-compose YAML file scp ./docker-compose.production.yml sadm@rubric-i.comp.nus.edu.sg:~/PM-Tracker/docker-compose.production.yml
```

## Steps

1. Move to project folder

```
cd PM-Tracker
```

2. Bring down current docker-compose system

```
docker-compose -f docker-compose.production.yml down
```

3. To free up space for updated containers, remove old images

```
Prune all unused images docker image prune -a # Check all images removed do
cker ps -a
```

4. Pull updated containers

```
docker-compose -f docker-compose.production.yml pull
```

5. Start up system

```
docker-compose -f docker-compose.production.yml up --detach
```

## For Mac users:

Reminder to include `platform: linux/amd64` in the `docker-compose.production.yml`, otherwise docker will build containers for MacOS by default

Turn off set environment variables on set-up in Terminal for ssh'ing

(<https://askubuntu.com/questions/599808/cannot-set-lc-ctype-to-default-locale-no->